

Main Memory

Operating System — Simple Study Notes

ICT 3101 — Operating System

1. What is Memory Management?

Memory management is a job of the operating system (OS). It controls the computer's main memory (RAM). It decides which program gets memory space and when.

What the OS does:

- It keeps a record of every part of memory — which part is used and which part is free.
- It checks how much memory each program (process) needs.
- It decides which process gets memory, and at what time.
- When a process finishes, the OS marks that memory as free again.

Simple picture: A process asks the OS for memory. The OS's "Memory Manager" finds free space and gives it to the process. When the process is not active for a long time, the OS may move it to the disk (this is called swapping). Later, it can bring the process back to memory (swap in).

2. Types of Memory Address

A program's address changes form as it moves from source code to running program. There are three types:

1) Symbolic Address

This is the address used in the program's source code — like variable names.

```
int x = 10;  
int y = x + 5;
```

Here, x and y are symbolic addresses because they are just variable names written by the programmer.

2) Relative Address

When the compiler compiles the code, it changes symbolic addresses into relative addresses (also called offsets — a distance from the start of the program).

Variable	Relative Address
x	100
y	104

3) Physical Address

When the program is loaded into RAM, the loader changes relative addresses into physical addresses. These are the real, actual locations in RAM.

$$\text{Physical Address} = \text{Base Address} + \text{Relative Address}$$

Example: Suppose the program is loaded starting at memory location 5000.

Variable	Relative Address	Physical Address
x	100	$5000 + 100 = 5100$
y	104	$5000 + 104 = 5104$

3. Base and Limit Registers

The OS uses two special registers to protect memory:

- **Base register** — stores the starting address of a process.
- **Limit register** — stores the size (range) of the process's address space.

Every time the CPU tries to access memory, the hardware checks the address:

1. Is the address $\geq$ base?
2. Is the address $<$ (base + limit)?

If both answers are yes, the access is allowed. If not, the CPU traps to the OS with an "addressing error" — the program tried to touch memory that is not its own.

Remember:

- *Base* = where the process starts in memory.
- *Limit* = how far the process is allowed to go.

4. Address Binding

Address binding means **connecting (mapping) a program's logical address to a real physical memory address**. This mapping is done by the MMU (Memory Management Unit) — a hardware part that changes logical addresses into physical addresses while the program runs.

An address can be bound (fixed) at three different times:

A) Compile Time

If we already know where the program will start in memory, the compiler can write the final (absolute) addresses directly.

- Example: Program always starts at address 1000. Compiler writes $x = 1020$, $y = 1030$ (fixed addresses).
- Problem: If the start address changes later (say to 5000), the whole program must be compiled again.

B) Load Time

If we do not know the start address at compile time, the compiler creates relocatable code (addresses written as offsets, not fixed numbers). The loader adds the real base address when the program is loaded.

- Example: Compiler generates $x = +20$, $y = +30$ (these are offsets).

$$\text{Physical Address} = \text{Base Address} + \text{Offset}$$

- If Base Address = 5000: $x = 5000 + 20 = 5020$, $y = 5000 + 30 = 5030$.

C) Execution Time (Run Time)

If a process can move to a different place in memory while it is running, the address binding must wait until run time. This method needs hardware support, like base and limit registers. Most modern operating systems use this method.

- Example: At first, Base = 10000 and Logical Address = 50, so Physical Address = 10050.
- Later, the OS moves the process: Base = 20000, Logical Address is still 50, so Physical Address = 20050.

The program does not need to change anything — only the base register value changes.

Binding Type	When Address is Fixed	Problem if Process Moves
Compile Time	Before running, by compiler	Must recompile program
Load Time	When program is loaded	Must reload program
Execution Time	While program is running	No problem — base register updates

5. From Source Code to a Running Program

Before a program can run, it passes through several stages. Each stage changes the program's form a little more, until it becomes something the CPU can actually execute.

1. Source Program — the code you write, example: `foo.c`
2. Compiler — checks the code and translates it into object code (example: `foo.o`). This is where symbolic addresses become relative addresses.
3. Linker (Linkage Editor) — combines your object code with library code (example: `libc.a`, which has functions like `printf()`, `scanf()`) and creates one Load Module (executable file, example: `a.out`).
4. Loader — takes the executable and loads it into main memory, and gets it ready to run. This is where relative addresses become physical addresses.
5. Machine Memory (Run Time) — the program now sits in RAM, next to the OS/Kernel and other programs, and can finally run.

Note: Steps 1–3 (source code to load module) happen at what we call "compile time" / "load time" — before the program starts running. Step 4 (loading) happens right before execution, and step 5 is execution itself.

6. MMU (Memory Management Unit) and Logical vs Physical Address

MMU is a hardware device. It changes a logical (virtual) address into a physical address, at the time the program runs.

- **Logical address** — the address used by the CPU/program. Every program thinks it starts at address 0.
- **Physical address** — the real address in RAM hardware.

The MMU uses a relocation register. It simply adds the relocation register's value to every logical address to get the physical address.

$$\text{Physical Address} = \text{Logical Address} + \text{Relocation Register Value}$$

Example: Relocation register = 14000. Logical address sent by CPU = 346.

$$\text{Physical Address} = 346 + 14000 = 14346$$

Key idea: The program never sees the real physical address — it only works with logical addresses. This makes the program easy to move in memory.

7. Memory Protection

Each process gets its own relocation register and limit register:

- **Relocation register** — stores the smallest physical address for that process.
- **Limit register** — stores the size of the logical address space.

When the CPU scheduler picks a process to run, the dispatcher loads the correct relocation and limit register values (this happens during a context switch). Every address made by the CPU is checked with these two registers before it can reach memory.

8. Dynamic Loading

Dynamic loading means: a part (routine/function) of a program is loaded into memory only when it is actually called/used, not before.

- Useful when the program is very large — we save memory by not loading unused parts.

```
main() {  
    f1();    // loaded only when called  
}  
f1() {
```

```
f2();    // loaded only when called
}
f2() {
    f3();    // loaded only when called
}
```

9. Dynamic Linking

Dynamic linking means: library functions (like `printf()`) are linked (connected) to a program while it runs, not before it runs.

How it works, step by step:

1. Instead of copying the full code of a library function into the program, the program keeps a small piece of code called a stub.
2. When the program runs and needs the function (example: `printf()`), the stub checks where that library function is located.
3. If the library is not already loaded, the OS loads it.
4. The stub gets the real address of the function.
5. The function runs.
6. Next time the same function is called, the program uses the same address directly (faster).

Static Linking vs Dynamic Linking

Static Linking	Dynamic Linking
Full code of <code>printf()</code> is copied into the program	Only a small stub is kept in the program
Executable file becomes larger	Executable file stays smaller
Library code is fixed inside the file	Library code (e.g. <code>libc.so</code>) is loaded only when needed

10. Overlays

Overlay is an old memory technique. It is used when a program is bigger than the available memory.

- Instead of loading the whole program at once, only the part that is needed right now is loaded.
- When a different part is needed, it replaces the old part in the same memory space.

Example

Available memory = 60 KB, Program size = 130 KB. The whole program cannot fit, so we break it into overlays.

Part	Size
Symbol Table (always in memory)	20 KB
Common Routines (always in memory)	30 KB
Overlay Driver (always in memory)	10 KB
Fixed part total	60 KB

There are also two big parts: Pass 1 (70 KB) and Pass 2 (80 KB). Only ONE of them is needed in memory at a time. When Pass 1 finishes work, it is removed, and Pass 2 is loaded into that same space.

11. Swapping

Swapping is a technique where a whole process is moved between main memory (RAM) and the disk (called the backing store).

- **Swap Out** — If a process is blocked for a long time (example: waiting for I/O), the OS removes it from RAM and stores it on disk.

- **Swap In** — When a process is ready to run again, the OS brings it back from disk into RAM.

Remember:

- *Swapping frees up RAM by moving inactive processes to disk, and bringing back active processes when needed.*

12. Memory Allocation — Fixed Sized Partition

This is one of the oldest and simplest ways to load more than one process into memory.

- Main memory is divided into fixed partitions (parts), equal or different in size.
- The OS itself stays in the first partition. Other partitions are used for user processes.
- Partitions cannot overlap.
- A process must fit completely inside one partition to run.

More Memory → More Processes → Higher CPU Utilization

CPU utilization means how much percent of time the CPU is actually doing useful work (not sitting idle).

$$\text{CPU Utilization} = 1 - P^n$$

Here, P = probability (chance) that one process is waiting for I/O (input/output), and n = number of processes that can fit in memory at the same time.

Example ($P = 0.7$, that is 70% chance a process waits for I/O)

Memory Size	Processes in Memory (n)	Chance CPU is Idle (P^n)	CPU Utilization ($1 - P^n$)
4 MB	1	$0.7^1 = 0.70$	$0.30 = 30\%$
8 MB	2	$0.7^2 = 0.49$	$0.51 = 51\%$
12 MB	3	$0.7^3 = 0.343$	$0.657 = 65.7\%$
16 MB	4	$0.7^4 = 0.2401$	$0.7599 = 76.0\%$
20 MB	5	$0.7^5 = 0.1681$	$0.8319 = 83.2\%$

Simple meaning: The CPU stays idle only when ALL processes in memory are waiting for I/O at the same time. So, more memory → more processes fit → less chance that all of them wait together → higher CPU utilization.

Problems in Fixed Partitioning

- **Internal Fragmentation** — When a process is smaller than its partition, the leftover space inside that partition is wasted. Example: a 4 MB partition holds a 3 MB process, so 1 MB is wasted.
- **External Fragmentation** — The unused space is scattered in different partitions. Even if the total free space is enough for a new process, it cannot be used because it is not together (not contiguous).

Example: Four partitions each waste 1 MB (total 4 MB free), but a new process of size 4 MB still cannot run because that free space is in four separate places, not one block.

- **Limit on process size** — A process cannot be bigger than the largest partition.
- **Low degree of multiprogramming** — "Degree of multiprogramming" means how many processes can stay in memory at the same time. In fixed partitioning, this number is limited and fixed, because partition sizes cannot change.

Remember:

- *Internal Fragmentation* → wasted space *INSIDE* a partition.
- *External Fragmentation* → free space scattered in *DIFFERENT* places.

13. Memory Allocation — Variable Sized (Dynamic) Partitioning

Here, partitions are not fixed. The OS gives a process exactly as much memory as it needs.

- The OS keeps a table (list) showing which parts of memory are free and which are used.
- At the start, all memory is one big free block, called a hole.
- When a process needs memory, the OS searches for a hole big enough for it.
- If found, the OS gives only the needed amount of memory to the process (not more).

When a process finishes and leaves memory, the OS can give that freed space to another ready process, matching the best size.

Advantages of Dynamic Partitioning

- **No Internal Fragmentation** — Partition size = process size exactly, so no space is wasted inside a partition.
- **No Size Limit** — A process is not limited by a fixed partition size.
- **Better Multiprogramming** — Since there is no wasted space, more processes can fit and run at the same time.

Disadvantages of Dynamic Partitioning

- **External Fragmentation** — Free spaces can still become scattered. Example: after some processes finish, we may have two small free blocks (1 MB and 1 MB) that cannot together load one 2 MB process, because they are not next to each other.
- **Complex Management** — Since partition sizes always change, the OS has extra work to track all partitions properly.

14. Compaction (Defragmentation)

Compaction is a technique that reduces external fragmentation. It moves all used (allocated) parts of memory together, and combines all free holes into one large block.

- This is also called defragmentation.
- After compaction, one big free space is available, so larger processes can now be loaded.

Example: Before compaction, free memory adds up to 7 MB, but it is split into small holes (2+1+3+1). A 6 MB process cannot be placed. After compaction, all 7 MB becomes one block, so a process up to that size can now fit.

15. Dynamic Storage Allocation — Tracking Free and Used Memory

The OS needs a way to remember which parts of memory are used and which are free. It usually uses a linked list for this. Each node (item) in the list represents one partition and stores three things:

1. Flag — shows if the partition is a hole (free) or has a process (used).
2. Start Address — where the partition begins.
3. End Address — where the partition ends.

Example list: [Process, 0, 99] → [Hole, 100, 199] → [Process, 200, 299] → [Hole, 300, 399]

16. Partitioning Algorithms (How to Choose a Hole)

When a process needs memory, the OS must choose which free hole to give it. There are three common methods:

Method	Rule	Note
First-Fit	Give the first hole that is big enough	Fastest search
Best-Fit	Give the smallest hole that is big enough (search the whole list)	Best use of memory, but leftover hole is very small and often wasted
Worst-Fit	Give the largest hole available (search the whole list)	Leaves the biggest leftover hole, which may be useful for future processes

Remember:

- *First-Fit* → fastest, picks the *FIRST* hole big enough.
- *Best-Fit* → picks the *SMALLEST* hole that fits (least memory waste, but tiny useless leftover).
- *Worst-Fit* → picks the *BIGGEST* hole (leaves a large usable leftover space).

Worked Example — Data

Suppose we have these jobs (processes) waiting, and these memory blocks (holes) are free:

Job	Size
Job 1	100k
Job 2	10k
Job 3	35k
Job 4	15k
Job 5	23k
Job 6	6k
Job 7	25k
Job 8	55k
Job 9	88k
Job 10	100k

Memory Block	Size
Block 1	50k
Block 2	200k
Block 3	70k
Block 4	115k
Block 5	15k

How to work it out: Go through the jobs one at a time (Job 1, Job 2, Job 3, ...). For each job, look at the current list of free blocks and apply the rule of the method you are using:

- **First-Fit** — place the job in the first free block (going block 1, 2, 3...) that is big enough.
- **Best-Fit** — check all free blocks, and place the job in the smallest block that is still big enough.
- **Worst-Fit** — check all free blocks, and place the job in the largest block available.

Once a block is filled, it is no longer free for the next job (unless a job later finishes and frees its block again). If a job does not fit in any free block, it must wait.

Note: With only 5 memory blocks (total $50+200+70+115+15 = 450\text{k}$) and 10 jobs, not every job can be placed at the same time — some jobs will have to wait until a block becomes free again.

17. Segmentation

Segmentation is a memory technique where a program is divided into different segments (parts) based on its logical structure — for example, one segment for the main function, one for data, one for utility functions, and so on.

- Each segment is a separate logical part of the program, and each can have a different size.
- Segments are loaded into non-contiguous memory (not necessarily next to each other), but each individual segment itself sits in one contiguous block.
- The OS keeps a Segment Map Table for every process. This table stores, for each segment: its size and its starting address in main memory.
- To find a piece of data, the CPU uses two values: the segment number and an offset (distance) inside that segment.

Example — Segment Map Table

Segment Number	Size	Memory Address (Start)
1	400	100
2	200	500
3	100	800

Meaning: Segment 1 has a size of 400 and starts at memory address 100. Segment 2 has a size of 200 and starts at address 500. And so on.

Quick Summary Table

Topic	One-Line Meaning
Memory Management	OS job of tracking and assigning memory to processes
Address Binding	Fixing a program's address at compile, load, or run time
MMU	Hardware that turns logical address into physical address
Base/Limit Register	Define start and size of a process's memory area
Dynamic Loading	Load a function only when it is called
Dynamic Linking	Link library code at run time using a stub
Overlays	Load only the needed part of a big program at a time
Swapping	Move whole processes between RAM and disk
Fixed Partitioning	Memory divided into fixed-size parts
Dynamic Partitioning	Memory given exactly as needed, size changes
Compaction	Combine free holes into one big block
First/Best/Worst-Fit	Rules to choose which hole to give a process
Segmentation	Program divided into logical parts (segments)